NATURAL LANGUAGE PROCESSING

Pretrained Word Embeddings

GloVe · FastText · BERT

From co-occurrence → subwords → contextual Transformers

Numan Aslam

LEARNING OUTCOMES

By the end, students should be able to…

1	**Explain pretraining**	Describe why previously learned vectors can be reused.
2	**Describe GloVe**	Use global word co-occurrence to explain static vectors.
3	**Describe FastText**	Explain character n-grams and rare or unseen words.
4	**Describe BERT**	Explain contextual token vectors and bidirectional attention.

Word2Vec was a starting point, not the end

1	**Word2Vec**	Learn local-context vectors through CBOW or Skip-Gram.
2	**GloVe**	Learn vectors from aggregated global co-occurrence counts.
3	**FastText**	Learn vectors using character-level pieces of each word.
4	**BERT**	Learn contextual token representations using a Transformer encoder.

Why borrow pretrained embeddings?

Your small dataset

500 course feedback comments

→

Large external corpus

Books · news · web pages (millions/billions of tokens)

→

Reusable representation

Vectors/checkpoint → your NLP application

Pretraining can save labeled data and training time—not eliminate all limitations.

Illustrative university feedback scenario; performance depends on model, domain and task.

Static vs contextual embeddings

Static · GloVe and FastText

“The bank approved my loan.”

“Sit beside the river bank.”

Lookup for bank → same word vector

Contextual · BERT

“The bank approved my loan.”

“Sit beside the river bank.”

Bank → different contextual token vectors

A word can have the same spelling but a different meaning in context.

ROADMAP

Three techniques, three learning ideas

1

GloVe

Which words occur together?

GLOBAL COUNTS

2

FastText

What letter pieces form a word?

SUBWORD PIECES

3

BERT

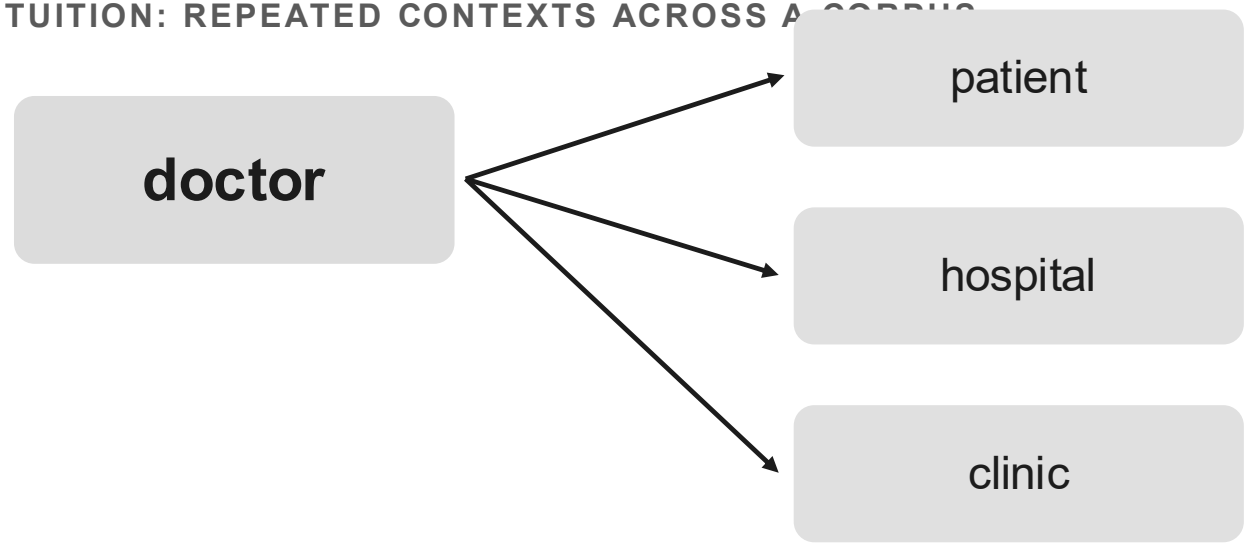
What does this token mean here?

BIDIRECTIONAL CONTEXT

Global co-occurrence → subword structure → contextual understanding

GloVe: learn from global word co-occurrence

INTUITION: REPEATED CONTEXTS ACROSS A CORPUS



Across many documents

Count patterns of nearby words—then learn compact vectors.

GloVe learns a static vector for each vocabulary word.

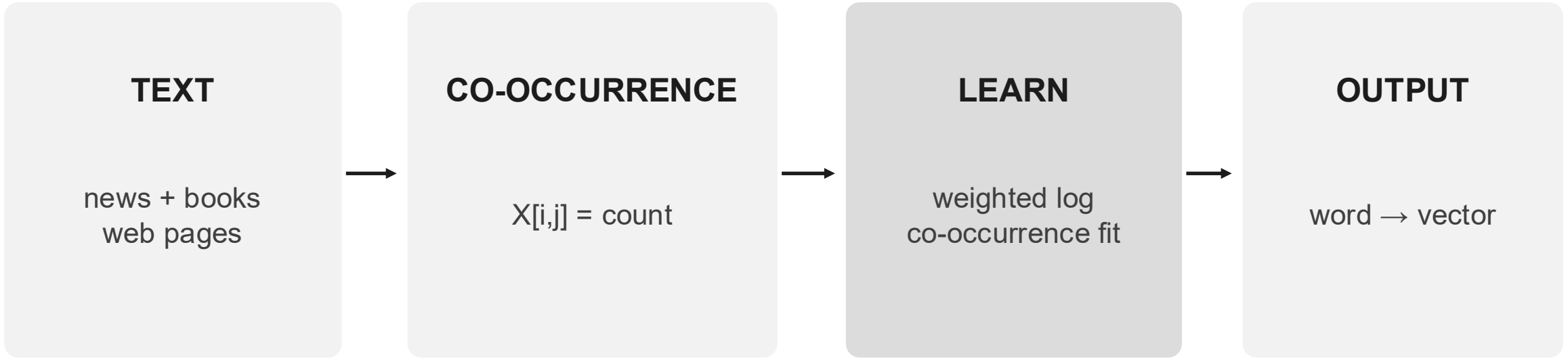
Turn nearby-word patterns into a matrix

ILLUSTRATIVE CO-OCCURRENCE COUNTS

focus / context	patient	clinic	cheese	delivery
doctor	42	37	1	0
nurse	33	26	0	1
pizza	0	0	40	29
burger	0	0	25	18

Illustrative integer counts for teaching, NOT values from any specific trained GloVe model.

Counts → statistical training → word vectors



Learned vectors reflect global word co-occurrence structure.

How is GloVe different from Skip-Gram?

Word2Vec · Skip-Gram

Training signal:
individual center → context examples

“doctor” predicts “patient”

Many separate word pairs

GloVe

Training signal:
aggregated word–word co-occurrence counts

X[doctor, patient] = 42 (toy)

A global count matrix

Both learn static vectors; the training signals differ.

What is the GloVe objective trying to fit?

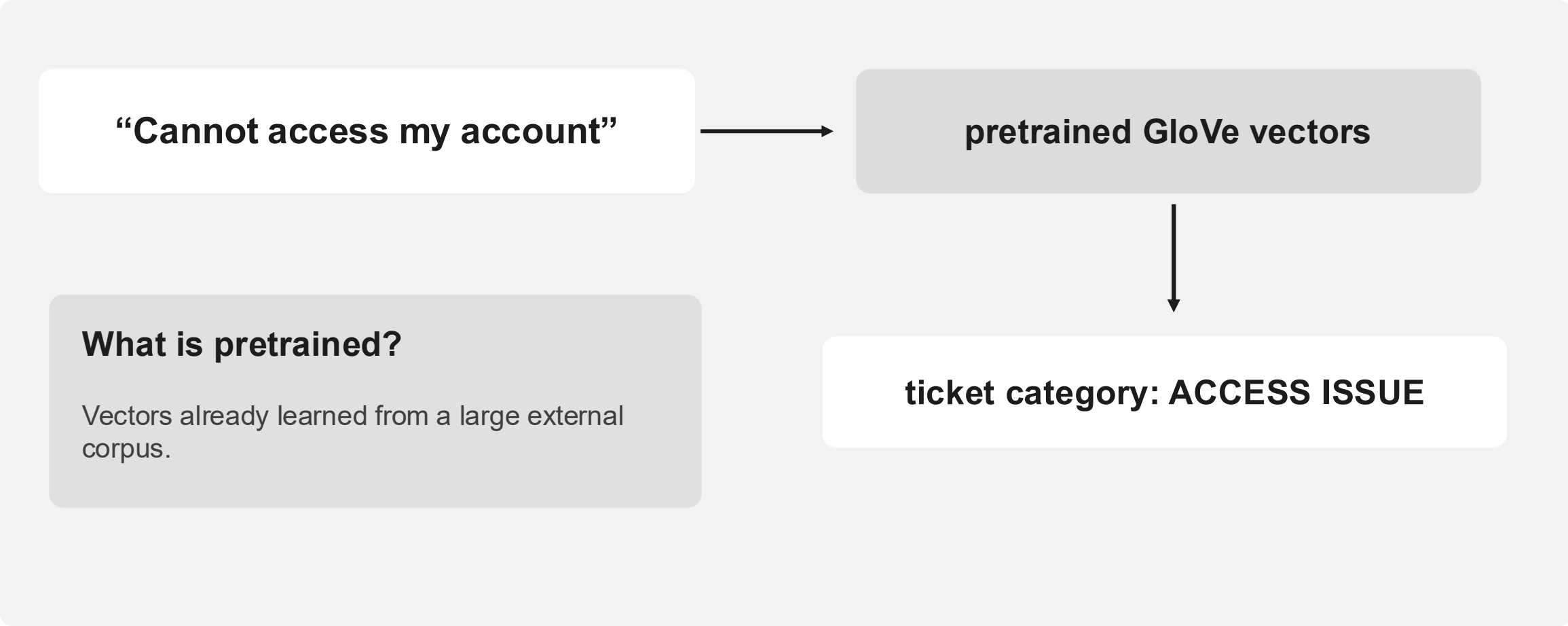
TRAIN THE LEARNED VECTORS TO EXPLAIN OBSERVED COUNTS

$$\text{word} \cdot \text{context} + \text{biases} \approx \log(\text{co-occurrence count})$$

Observed	Predicted	Update
X[doctor, patient]	vector score	reduce weighted error

Conceptual rendering of GloVe's weighted least-squares, log-co-occurrence objective.

Reuse pretrained GloVe in a student project



Application is illustrative; a classifier must still be built/trained and evaluated.

Where is GloVe strong? Where does it struggle?

Why use GloVe?

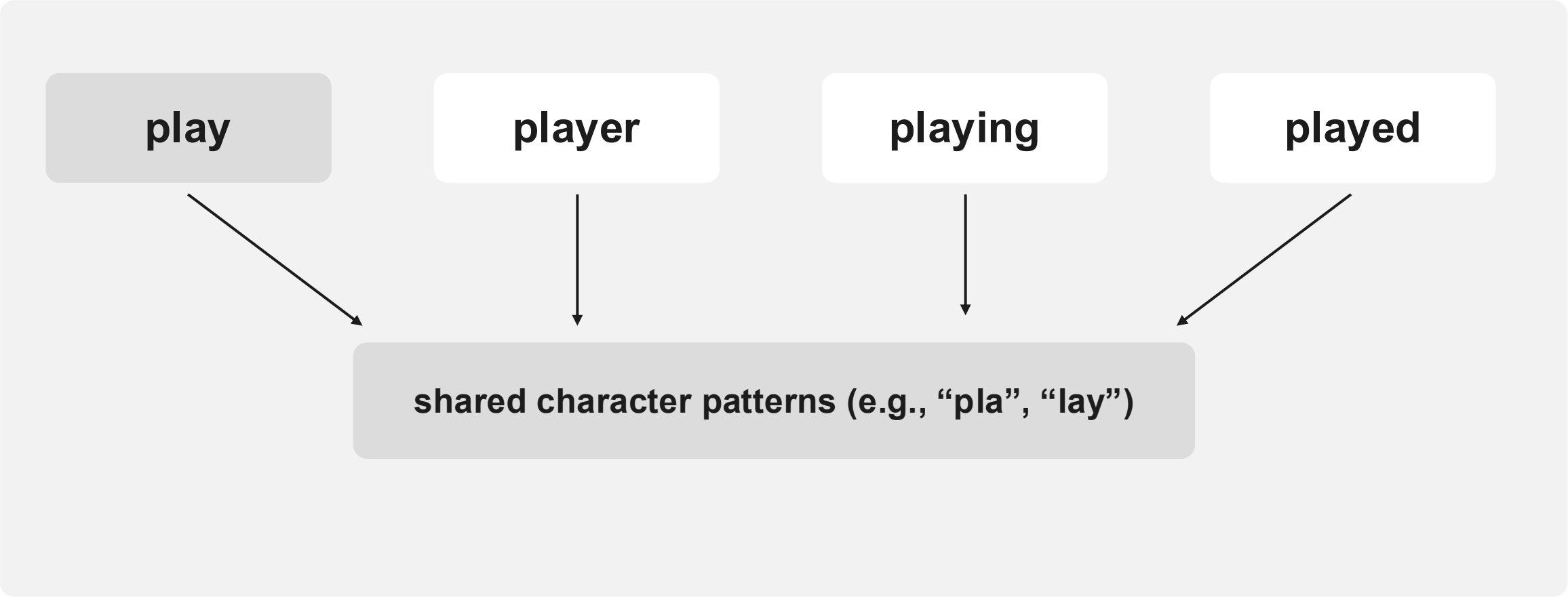
- Compact, reusable vectors
- Useful baseline for word similarity
- Easy input features for smaller models

Main limitations

- One vector per vocabulary word
- Unknown words need a fallback
- Corpus bias / domain mismatch

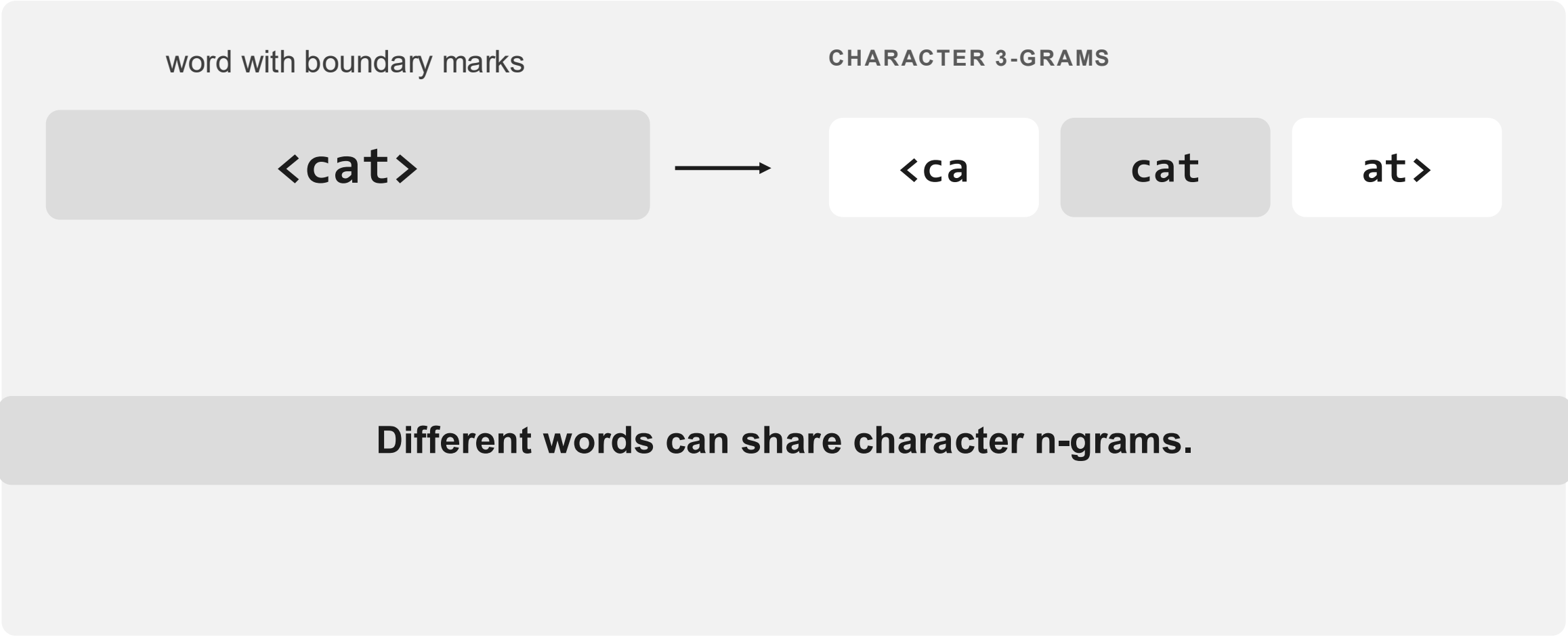
GloVe knows global co-occurrence patterns—not a word's current sentence.

FastText: look inside the word



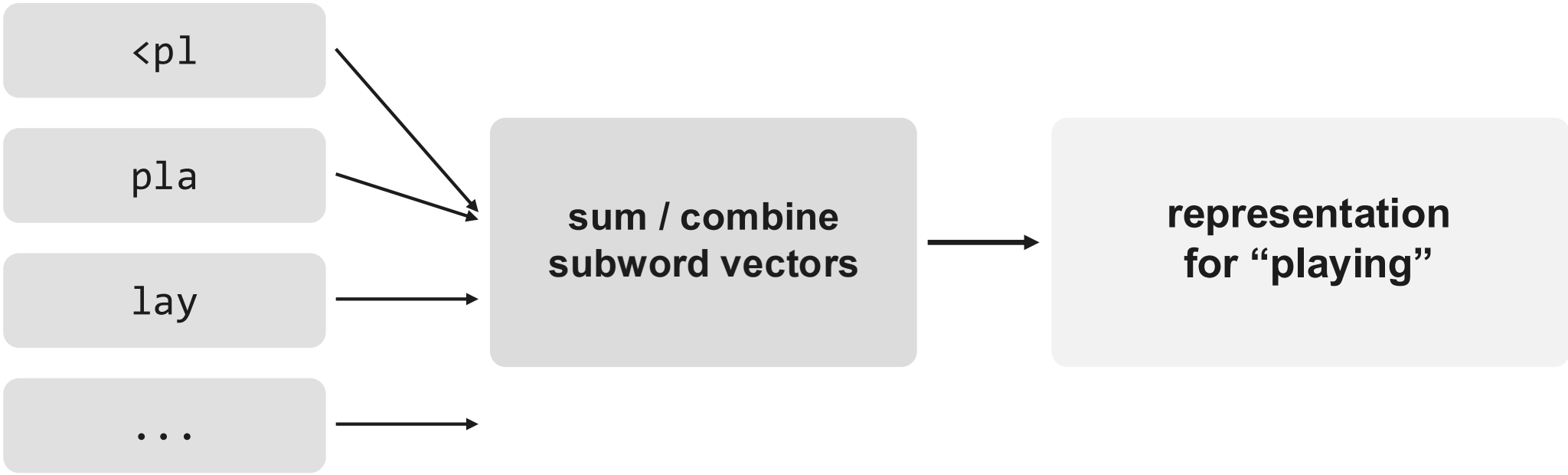
Shared spelling helps, but it does not guarantee equal or correct meanings.

What are character n-grams?



Boundary notation and 3-gram example are simplified for teaching.

Several character-piece vectors make one word vector



Shown: illustrative subset of subwords; actual n-gram inventory is larger.

Why “play”, “playing” and “played” benefit

Whole-word approach

play → lookup vector A
playing → lookup vector B
played → lookup vector C

No explicit shared letter-piece structure

FastText subword approach

play → pieces of “play”
playing → overlapping pieces
played → overlapping pieces

Shared parameters can transfer patterns

FastText explicitly reuses spelling patterns across word forms.

What if a word was missing during training?

“resturant”



GloVe-style lookup

Unknown token → no vector found in the downloaded vocabulary.

FastText-style model

Known character pieces → possible approximate vector.

spelling cues, not a guaranteed spelling correction

Some .vec / plain text exports are only fixed word lookup tables and do not support OOV composition.

Customer search with varied spellings

ILLUSTRATIVE INPUTS

wireless earphones

wireless earphone

wireles earphones



subword-based representations

Useful in noisy text—but it must be evaluated with realistic errors.

How do we reuse pretrained FastText?

1	**Choose language**	Select a model compatible with project text.
2	**Download checkpoint**	Use a pretrained FastText .bin file to preserve subwords.
3	**Load + query**	Call get_word_vector(word) for known/unseen words.
4	**Evaluate**	Test quality on the project’s real examples.

FastText: benefits and limitations

Benefits

- Learns shared character patterns
- Can handle many unseen word forms
- Useful for morphologically rich/noisy text

Limitations

- Usually one static representation per word
- Similar spelling may mislead
- Subword support depends on saved format

FastText adds word structure, but not full sentence-dependent meaning.

BERT: the sentence changes the representation

SENTENCE 1

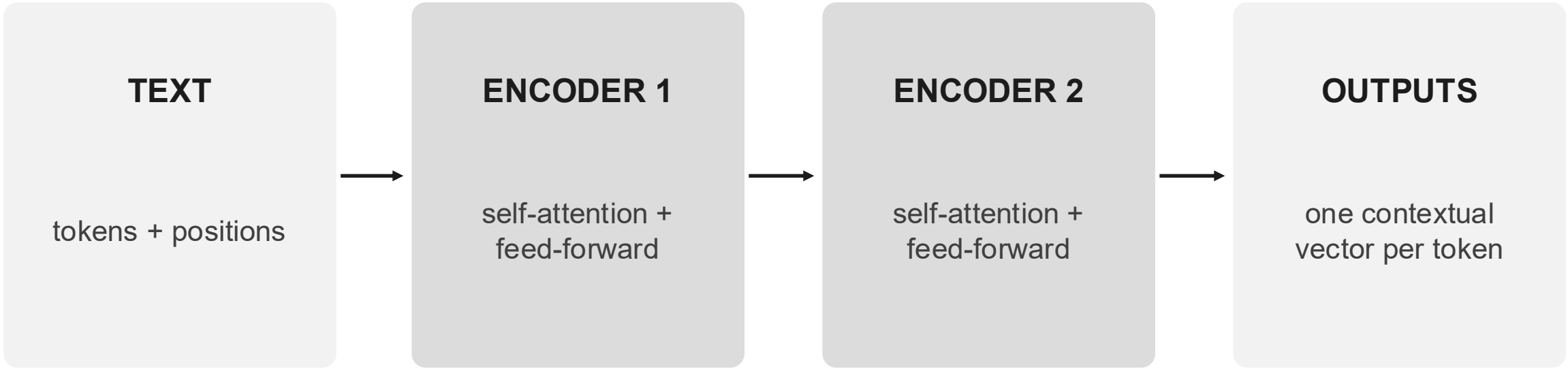
“The bank approved my loan.”

SENTENCE 2

“We sat on the river bank.”

In BERT, “bank” receives a context-sensitive representation.

The Transformer encoder is the core



BERT is a deep encoder model—not merely a table of word vectors.

BERT often splits words into subword tokens

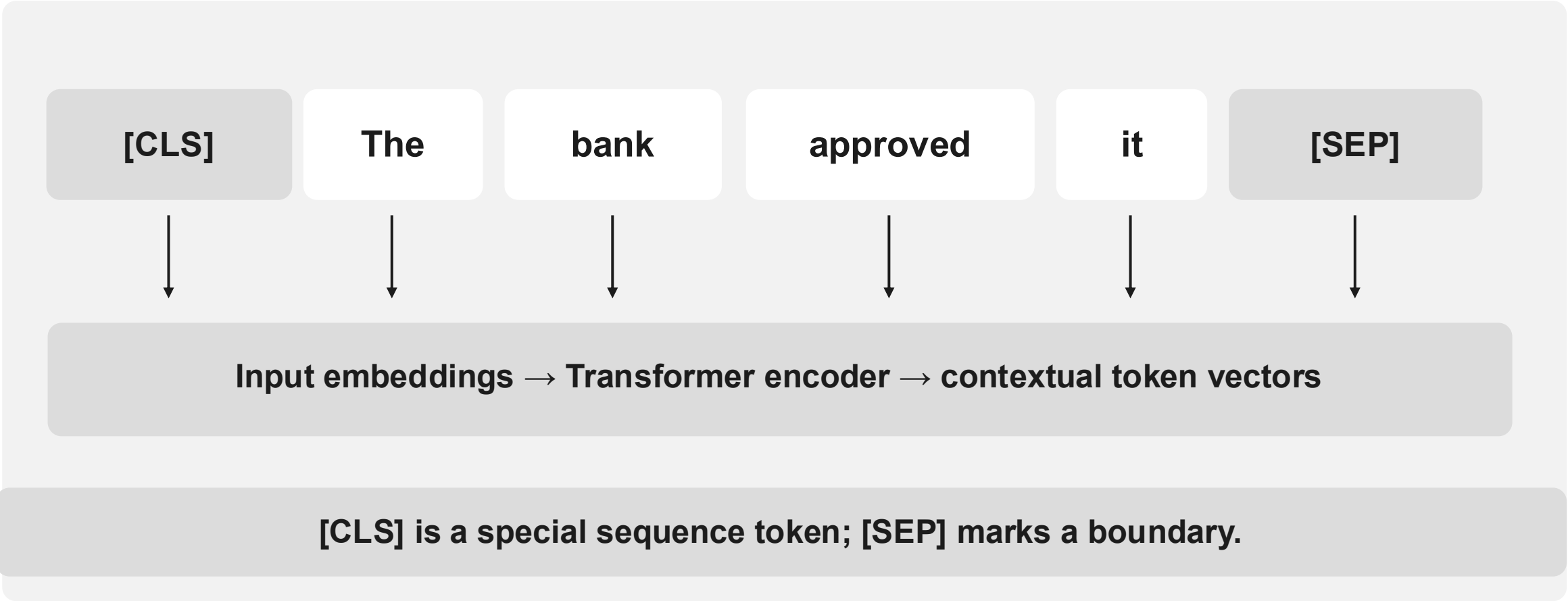
ILLUSTRATIVE WORDPIECE SPLIT



word → subword token IDs → contextual Transformer output

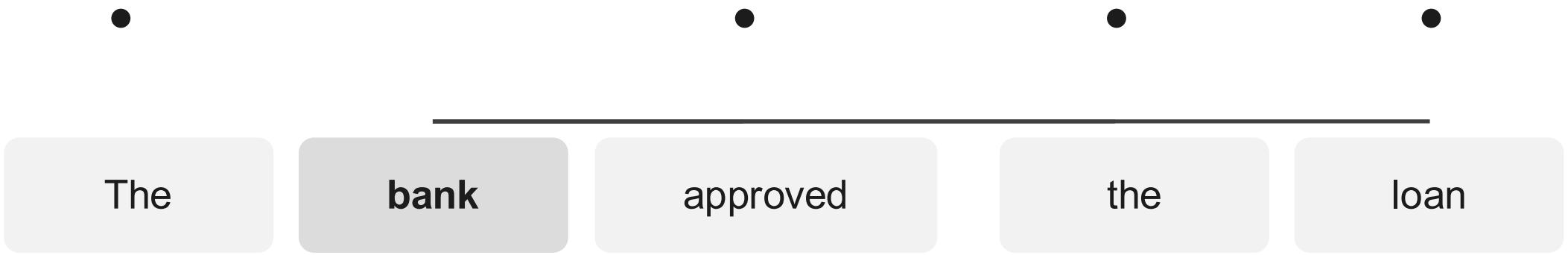
Illustrative splitting only: exact WordPiece segments depend on model/tokenizer vocabulary.

Special tokens give structure to input



Self-attention connects tokens across a sentence

ILLUSTRATIVE ATTENTION CONNECTIONS

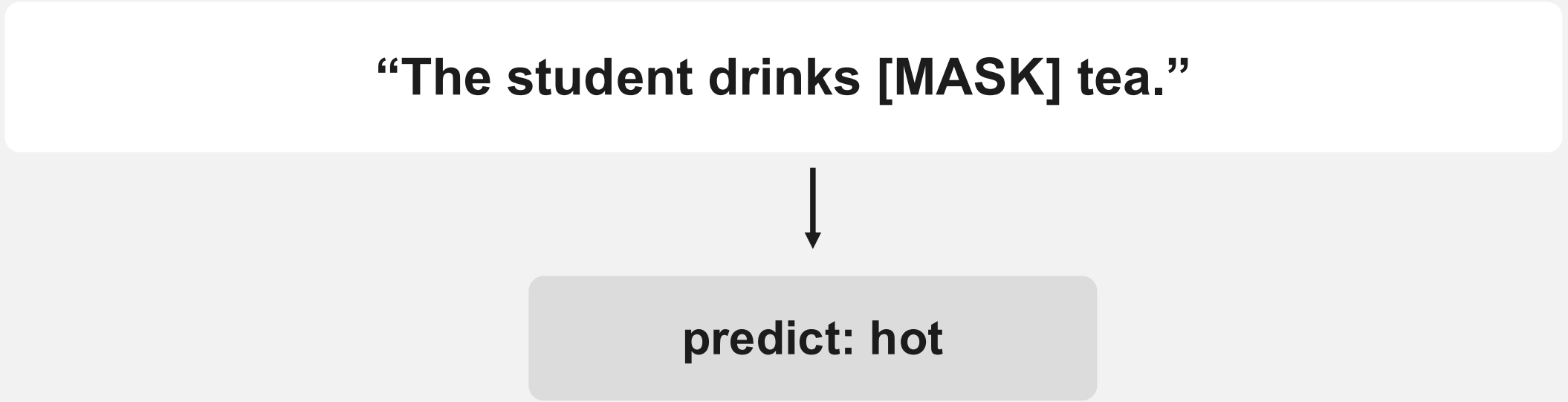


BANK uses information from surrounding tokens

Simplified connection sketch—not measured attention weights from a trained BERT model.

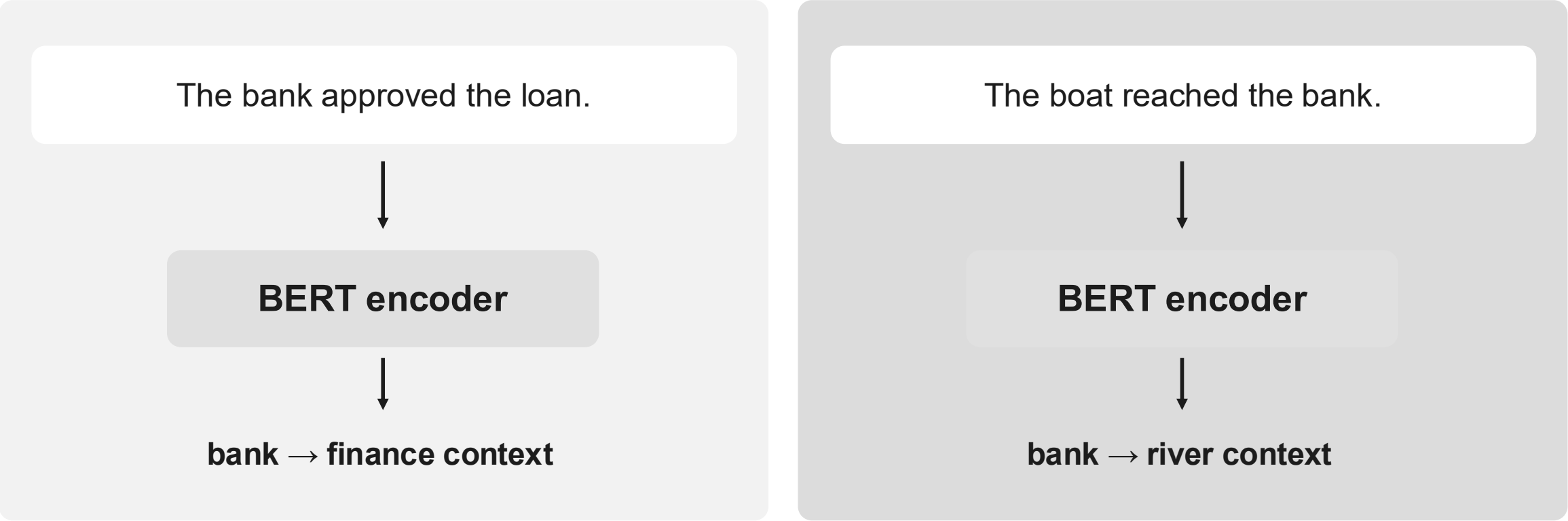
Masked language modeling: predict a hidden token

PRETRAINING-TASK ILLUSTRATION



The mask illustration teaches the objective; original BERT selects 15% of token positions, with replacement rules.

Why the same word can have two vectors



Vectors differ by input context and chosen layer; labels summarize intended senses only.

What can a pretrained BERT model do?

Sentiment / intent

“The app keeps crashing.”

negative / bug report

Named entities

“Aisha joined Acme in Lahore.”

person · organization · city

Question answering

“When was the policy revised?”

find answer span in passage

BERT can be fine-tuned for tasks beyond word similarity.

Reuse BERT: freeze its features or fine-tune?

Feature extraction

Pretrained BERT (frozen)
→ contextual vectors
→ small downstream model

Less parameter training

Fine-tuning

Pretrained BERT (trainable)
+ task head + labeled examples
→ update parameters

More task adaptation / compute

Choose the reuse strategy based on data, task, and computing budget.

Python: loading GloVe and FastText vectors

GloVe · Gensim

```
import gensim.downloader as api
glove = api.load(
  'glove-wiki-gigaword-50')
print(glove.most_similar('doctor',
                         topn=3))
```

FastText · official Python package

```
import fasttext
ft = fasttext.load_model(
  'cc.en.300.bin')
v = ft.get_word_vector('playing')
print(v.shape)
```

Requires: pip install gensim fasttext ; models are downloaded separately / on first call.

Python: extract contextual BERT vectors

```
from transformers import AutoTokenizer, AutoModel
import torch
tok = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModel.from_pretrained('bert-base-uncased')
inputs = tok('The bank approved the loan.', return_tensors='pt')
with torch.no_grad():
    outputs = model(**inputs)
print(outputs.last_hidden_state.shape)  # [1, tokens, 768]
```

Requires: pip install transformers torch ; a model download and memory for inference.

SIDE-BY-SIDE COMPARISON

GloVe vs FastText vs BERT

Dimension	GloVe	FastText	BERT
Main idea	global co-occurrence	character subwords	context + attention
Typical output	static word vector	static word vector	contextual token vector
Unknown word	no direct lookup	subwords may help	subword tokenizer
Same word, new context	same vector	same vector	different vector
Compute to reuse	often low	often low–moderate	typically higher

“Compute” is relative and depends on vocabulary, sequence lengths, hardware, checkpoint sizes and workload.

Which method would you choose?

1. **A · Small CPU project** — Pretrained word lookup for a simple sentiment baseline.

2. **B · Noisy text** — Classify many inflected words and spelling variants.

3. **C · Ambiguous meanings** — Tell “bank loan” from “river bank” in full sentences.

The correct choice depends on your task—not just model popularity.

CLASS ACTIVITY · 5 MINUTES

Try it: discuss in groups

1	**Q1 · Mechanism**	What extra information does FastText use beyond whole words?
2	**Q2 · Meaning**	Which method changes “bank” when the sentence changes?
3	**Q3 · Limitations**	Why is a FastText vector for a misspelling not guaranteed correct?

Ask students to justify each answer with a one-sentence example.

Check your reasoning

1	**A1 · FastText**	Character n-gram subword information.
2	**A2 · BERT**	The bank token depends on its surrounding sentence.
3	**A3 · Caution**	Spelling similarity does not guarantee semantic correctness.

Use appropriate baselines and test on real project data.

Remember the three questions

1	**GloVe**	Which words occur near one another across the corpus?
2	**FastText**	Which character pieces form the written word?
3	**BERT**	What does this token mean in this sentence?

Pretrained does not mean perfect; evaluate models on your own task.

FURTHER READING

Original papers and practical resources

GloVe · Pennington et al. (2014)

https://nlp.stanford.edu/projects/glove/

FastText · Bojanowski et al. (2017)

https://aclanthology.org/Q17-1010/

BERT · Devlin et al. (2019)

https://aclanthology.org/N19-1423/

Hugging Face · BERT docs

https://huggingface.co/docs/transformers/model_doc/bert